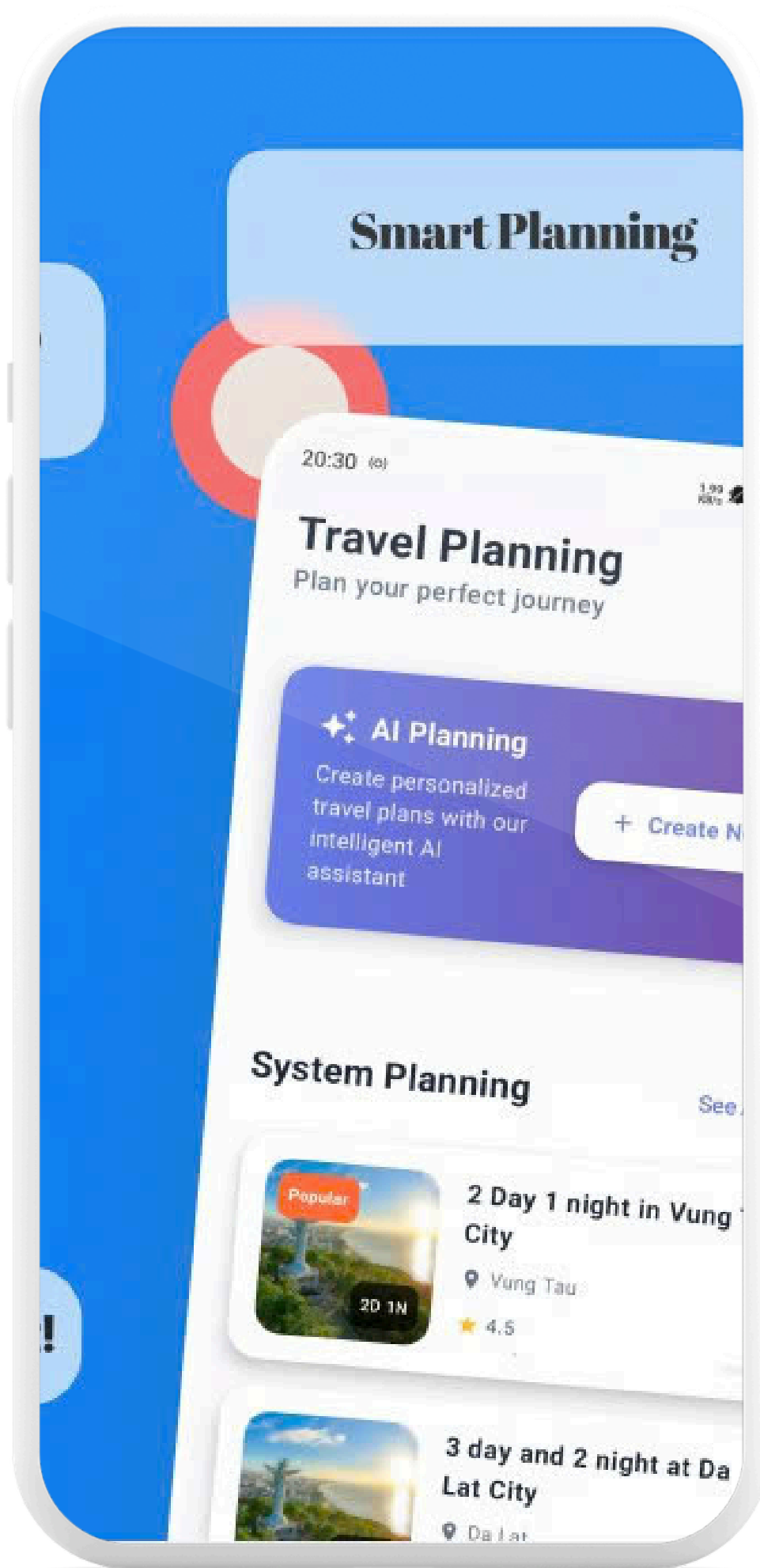


Kiến trúc tổng thể hệ thống



App Mobile

- Android: Jetpack Compose (Kotlin)

Classification model

- Yolo v8
- TensorFlow Lite
- Kaggle

Sercurity

- JWT , Device Id, Secret Key, SSL

Web service

- Backend chính: NestJS
- Schedule API: NestJS
- Payment: SePay
- Mail Service
- FCM service(Firebase - Push notitfcation)
- Storage: Cloud flare R2
- Gemini API.
- Database: postgresql

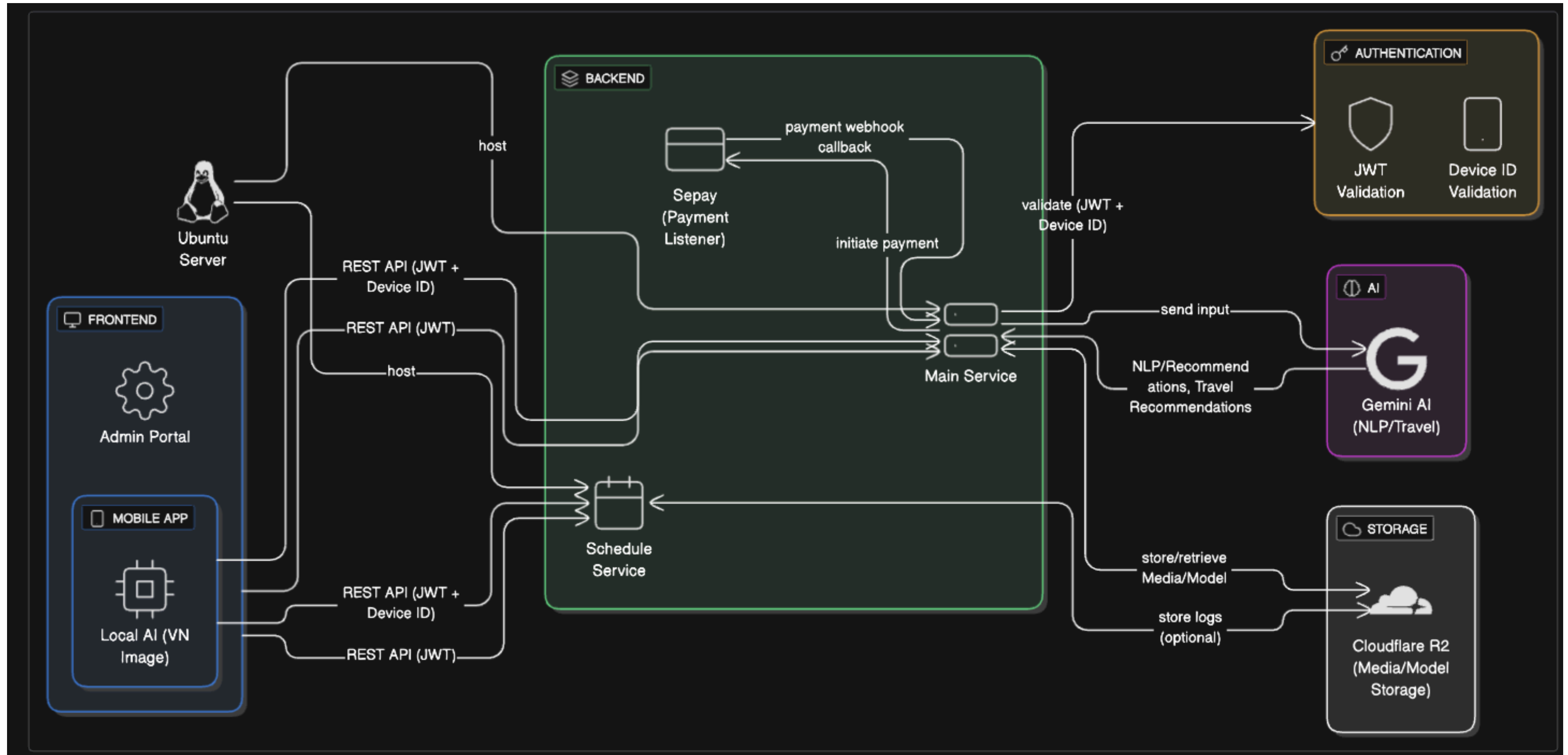
Admin portal

- Công nghệ: ReactJs

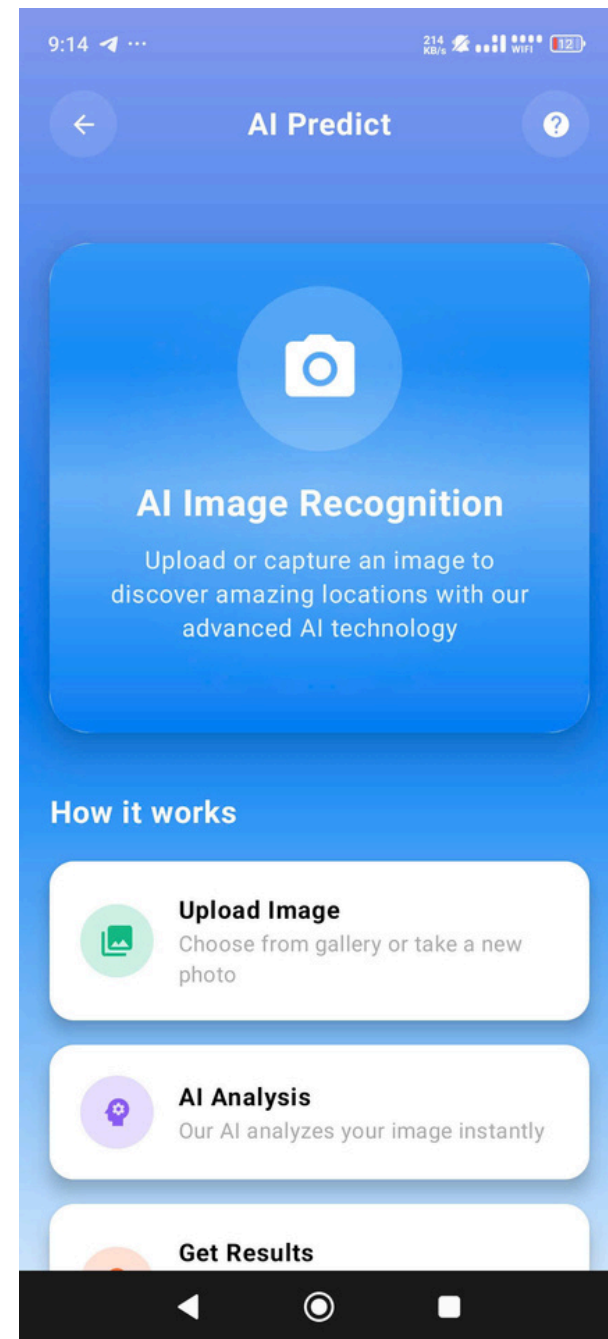
Deploy System

- Cloud + docker.
 - Database + Server: Deploy trên cloud server ubuntu
- CI/CD: Github Actions

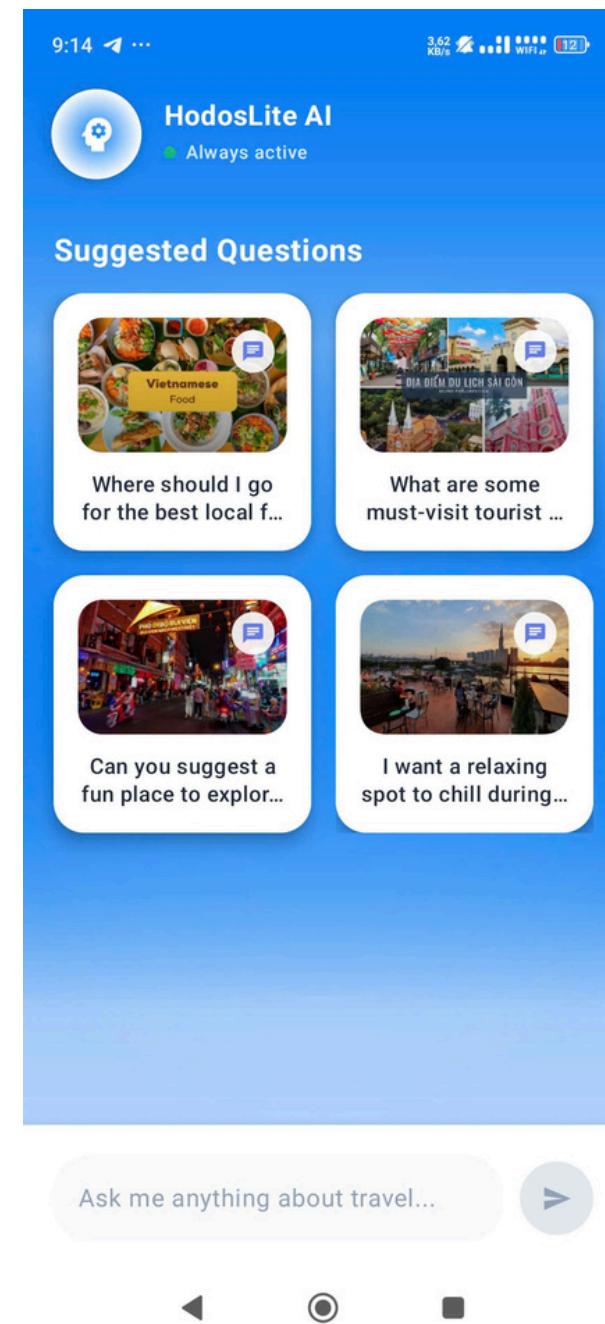
Sơ đồ Kiến trúc tổng thể hệ thống



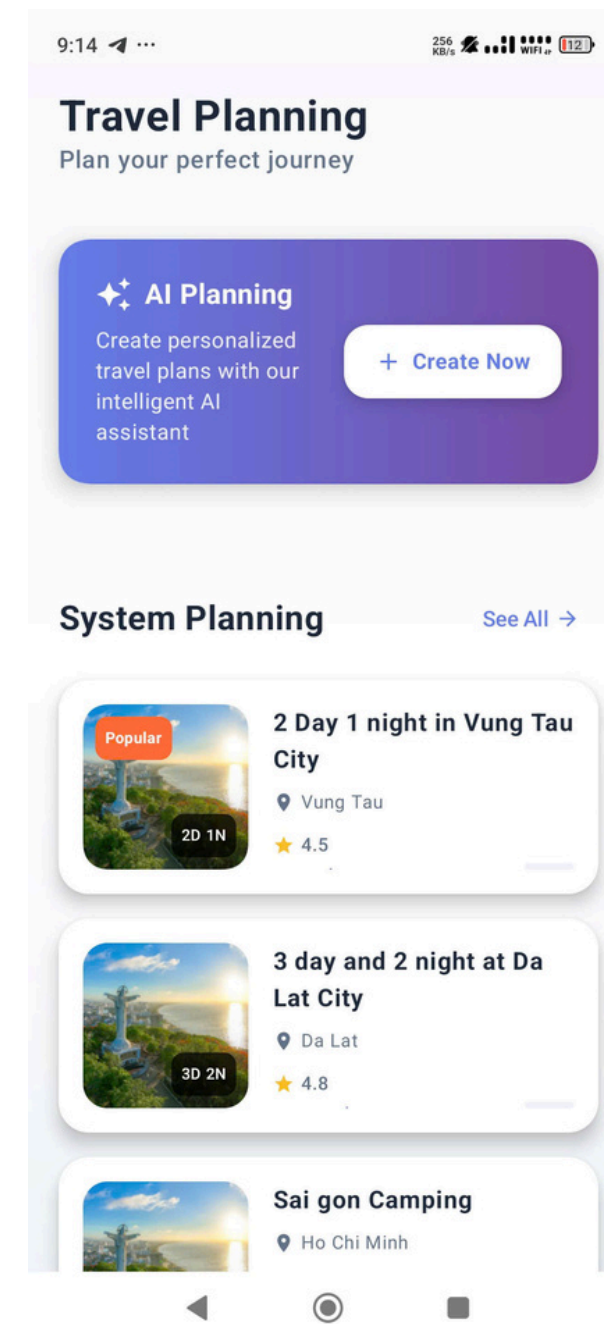
Giới thiệu các tính năng



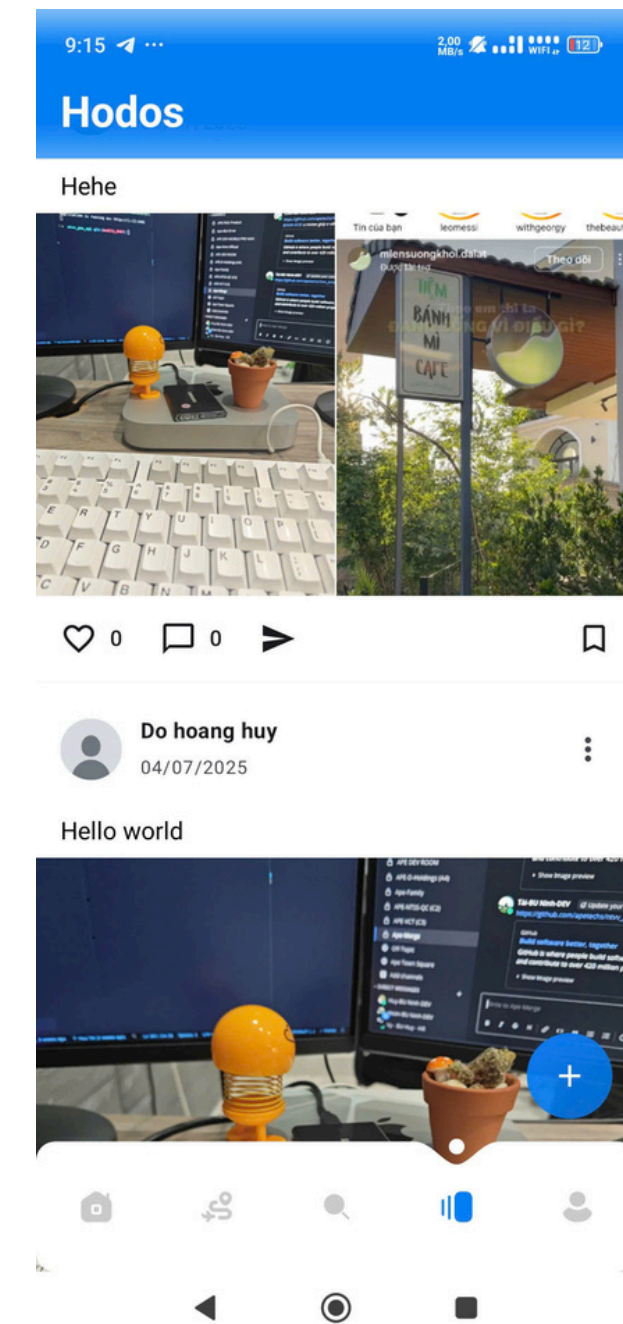
Nhận dạng hình ảnh



ChatBot AI



Lên kế hoạch



Chia sẻ khoảng khắc

Phát triển mô hình nhận dạng địa điểm và món ăn cho chức năng nhận dạng

Tập dữ liệu địa điểm

Tập dữ liệu địa điểm có tổng cộng **7.810 hình ảnh** được chia thành **27 lớp**.

Train: **5.670 hình** Test: **1.620**

hình Validation: **810 hình**

=> Dữ liệu được tự thu thập từ Google Image, Youtube, ảnh thực tế.

Tập dữ liệu món ăn

Tập dữ liệu món ăn có tổng cộng **12.840 hình ảnh** chia thành **21 lớp**.

Train: **7.740 hình** Test: **2.080**

hình Validation: **1.040 hình**

=> Dữ liệu được lấy từ tập dữ liệu 30VNFoods khoảng 10.000 hình và tự thu thập khoảng 2.800 hình từ Google Image, Youtube, ảnh thực tế.

Một số hình ảnh tự thu thập



Bùi Viện



Thành phố



Chùa
Ngọc Hoàng



Chợ

Đề xuất: Mô hình sử dụng



Bảng 3.1: Thông số các mô hình đã tìm hiểu

Pretrained-model	Kích thước (MB)	Độ sâu	Số lượng tham số
MobileNetV2	14	105	3.5 Triệu
ResNet152V2	232	307	60.4 Triệu
InceptionV3	92	189	23.9 Triệu
Xception	88	81	22.9 Triệu
InceptionResNetV2	215	449	55.9 Triệu
YOLOv8n	2.7	224	1.4 Triệu

Nguồn: <https://keras.io/apilapplications/>

Kết quả: Mô hình



Các kết quả đánh giá các mô hình

Bảng 4.4 Bảng so sánh và kết quả chạy thử nghiệm các model

Model	Accuracy	F1_Score
MobileNetV2	0.9262	0.9266
ResNet152V2	0.9300	0.9302
InceptionV3	0.8954	0.8951
InceptionResNetV2	0.8665	0.8657
Xception	0.9005	0.9007
YOLOv8	0.9712	0.9711

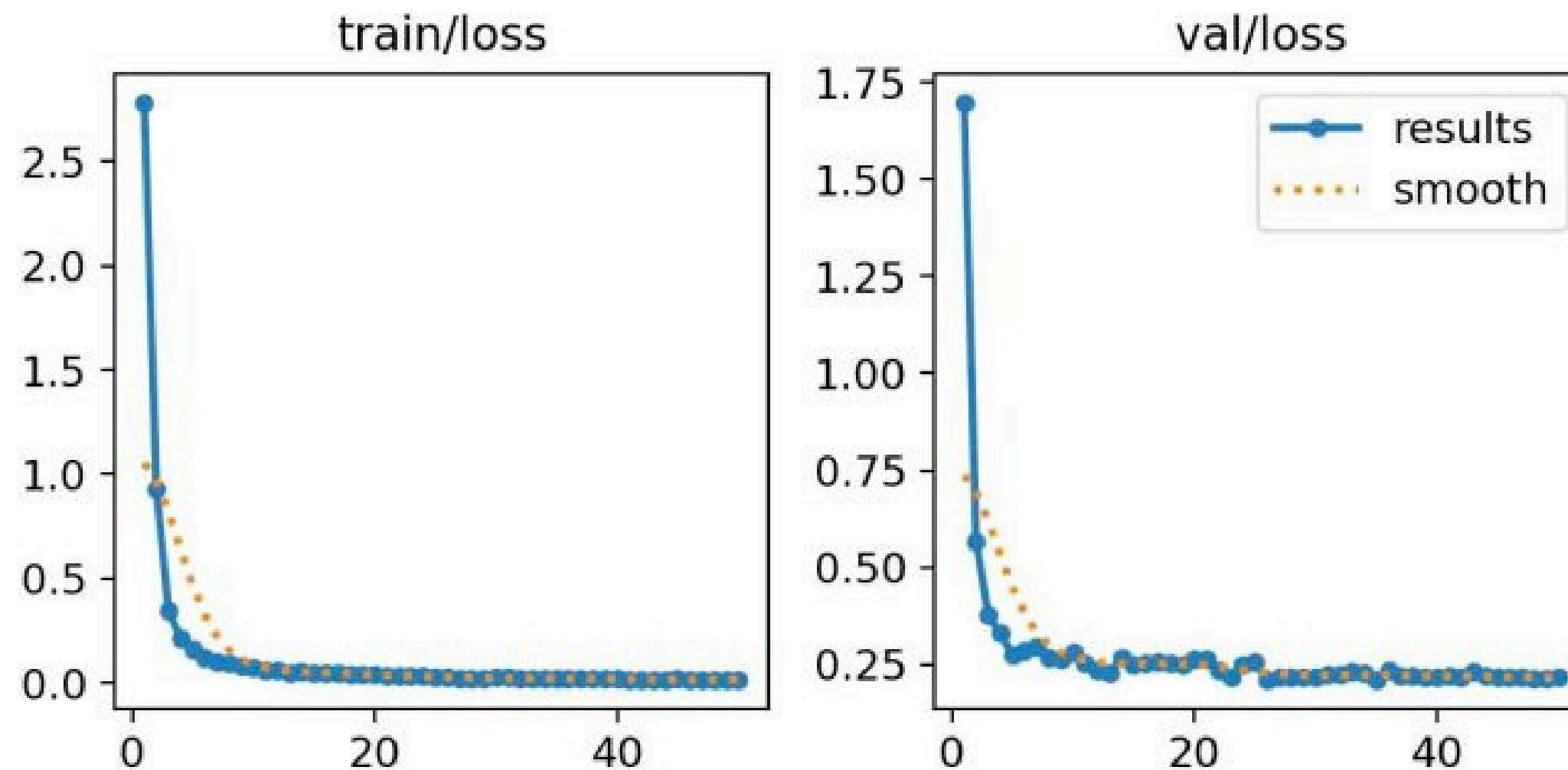
➡ **YOLOv8** là pretrained-model **thích hợp nhất** với tập dữ liệu hiện tại.

Kết quả: Mô hình sử dụng



Các kết quả đánh giá các mô hình

Hình 4.1: Loss và Accuracy của quá trình huấn luyện mô hình nhận dạng



Kết quả: Mô hình sử dụng



Một số kết quả nhận diện với hình ảnh



1/1 [=====] - 3s 3s/step
Bao_Tang_Thanh_Ph

Hình 4.5: Mẫu dự đoán 1



1/1 [=====] - 3s 3s/step
Thao_Cam_Vien

Hình 4.6: Mẫu dự đoán 2

Kết quả: Mô hình sử dụng



Một số kết quả nhận diện với hình ảnh



1/1 [=====] - 5s 5s/step
Pho

Hình 4.7: Mẫu dữ đoán 3

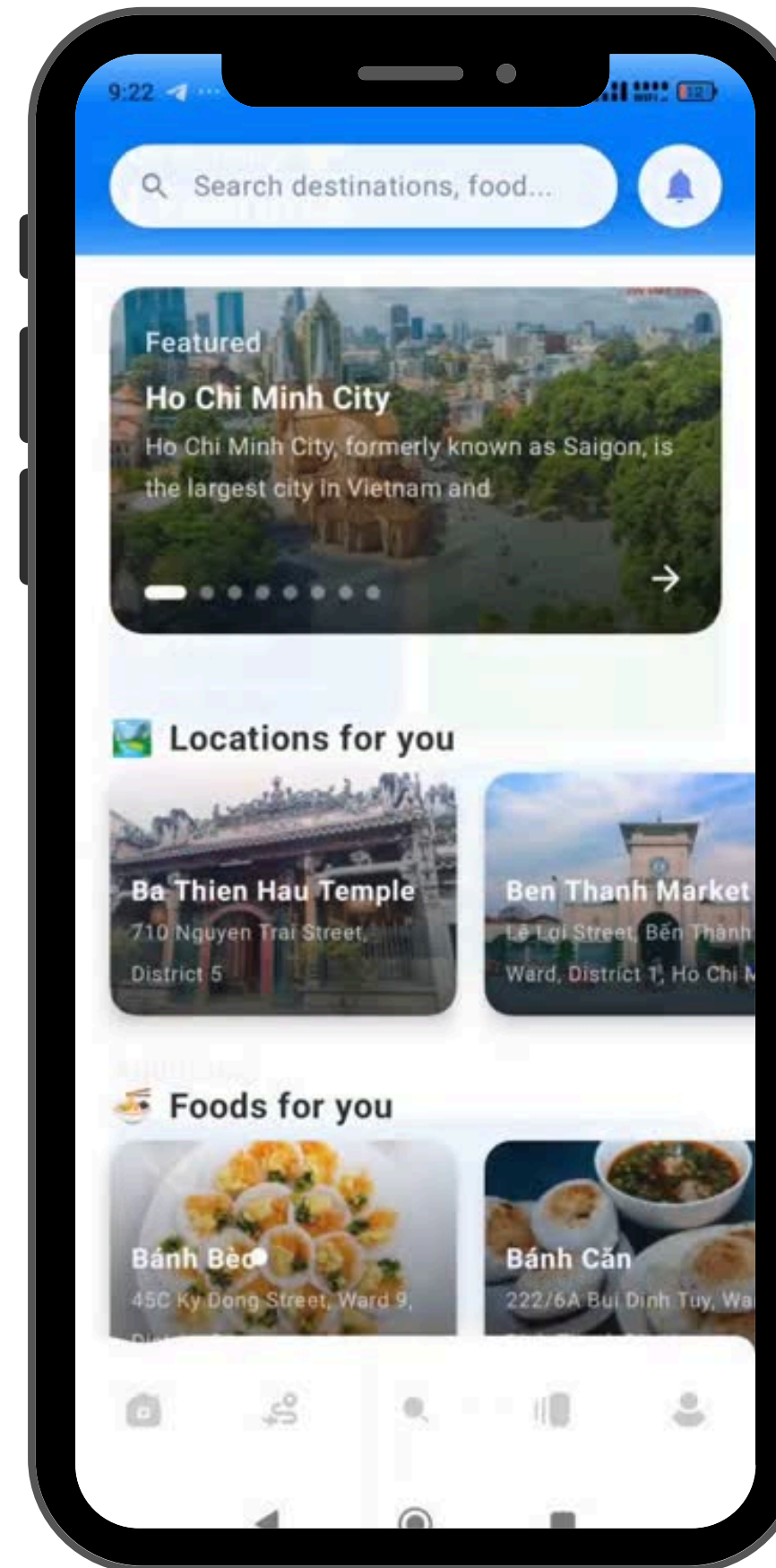


1/1 [=====] - 3s 3s/step
Hu_Tieu

Hình 4.8: Mẫu dự đoán 4

Tích hợp mô hình vào ứng dụng di động

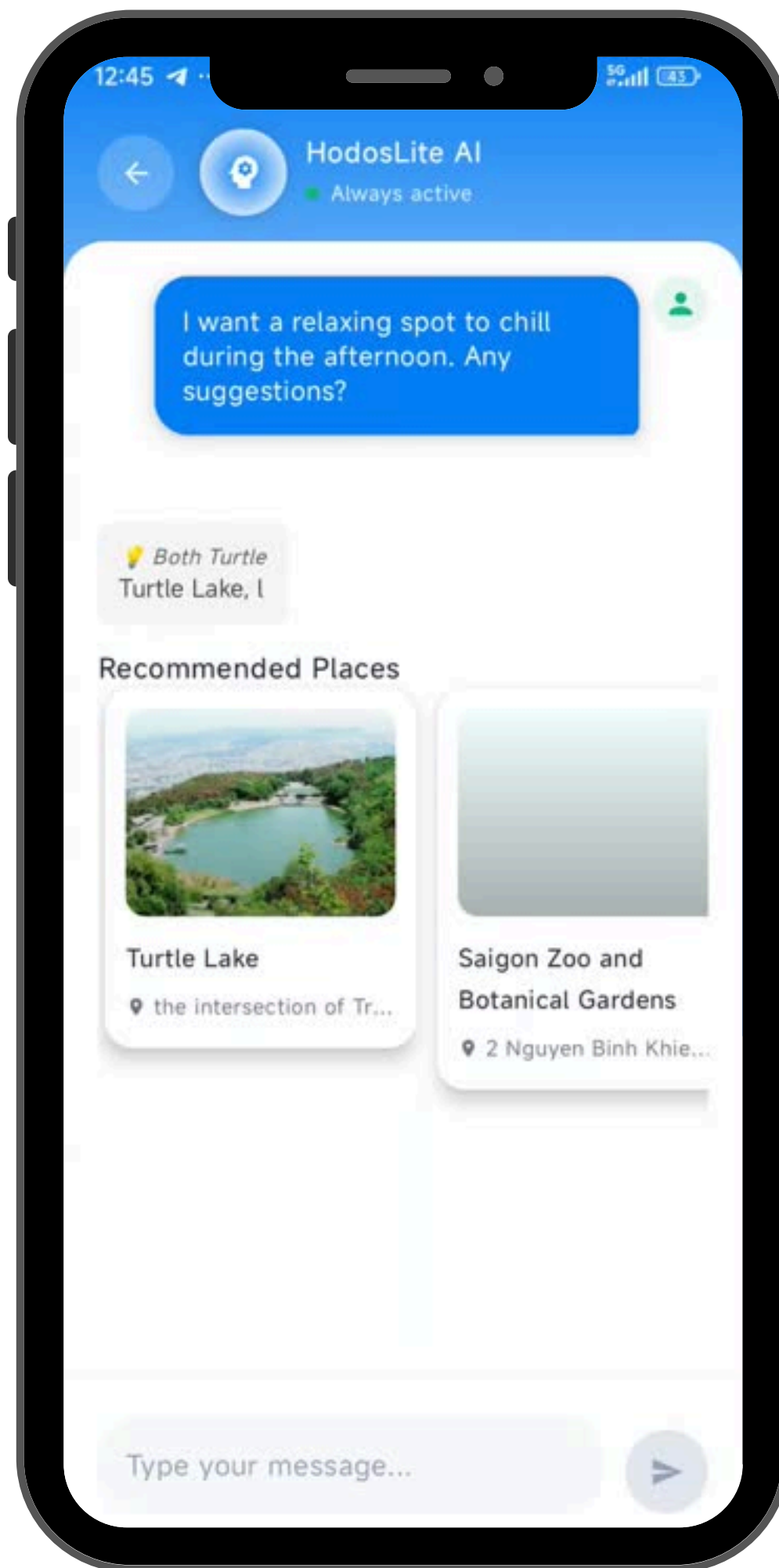
- Người dùng chụp ảnh hoặc chọn ảnh từ thư viện
- Ứng dụng Android sử dụng mô hình đã được tích hợp.
- AI model xử lý ảnh trực tiếp trên điện thoại
- Kết quả nhãn (label + độ chính xác) được trả về ngay
- App hiển thị nhãn
- Gợi ý hành động tiếp theo (ví dụ: mở địa điểm, gợi ý món ăn)



Khi có kết nối mạng



Khi không có kết nối mạng



ChatBot AI

Mục tiêu tính năng

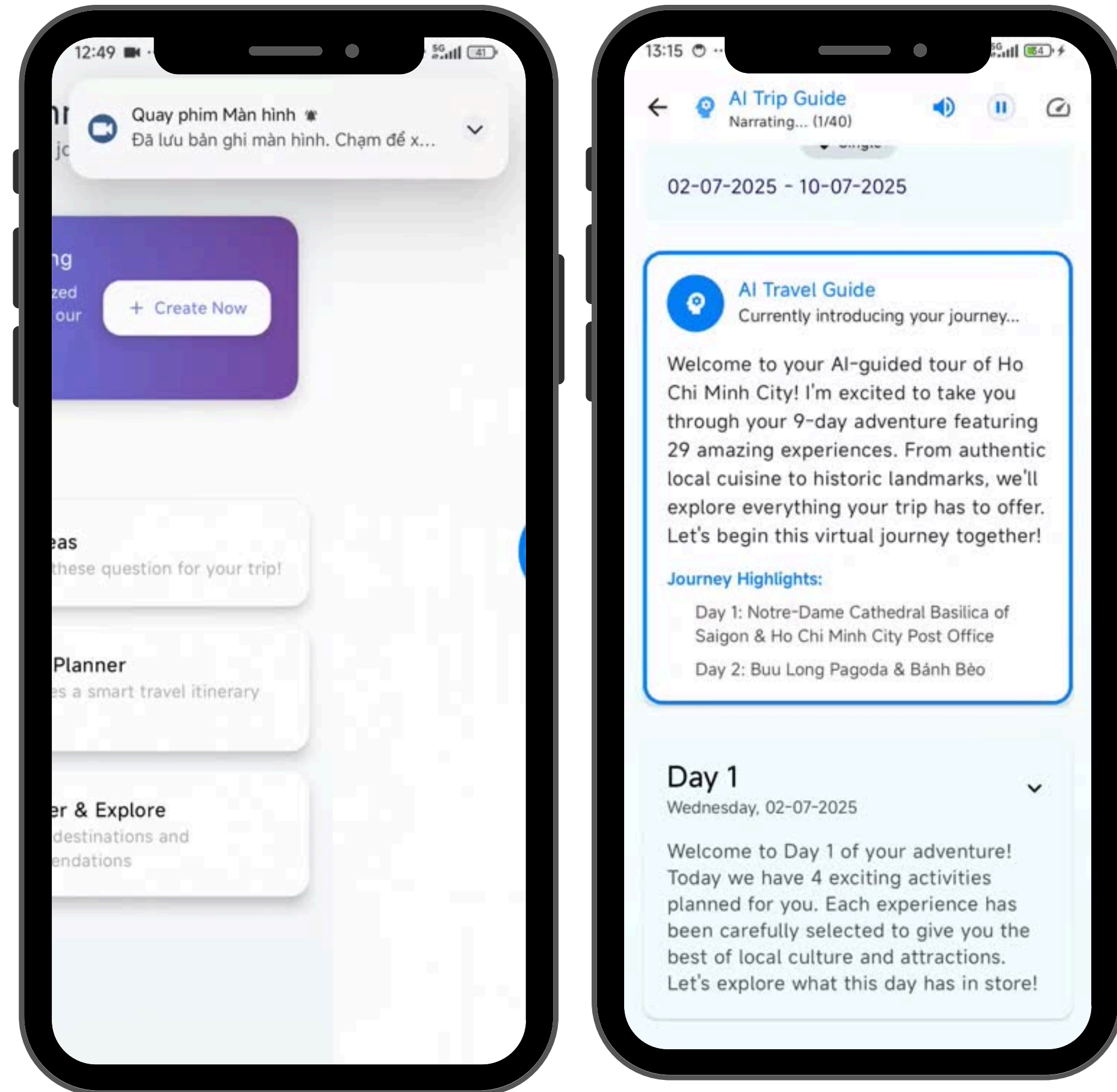
Cho phép người dùng chat trực tiếp với AI để:

- Hỏi gợi ý địa điểm theo nhu cầu, sở thích
- Nhận được đề xuất địa điểm phù hợp
- Kèm theo giải thích lý do chọn địa điểm đó

Đề xuất kế hoạch du lịch

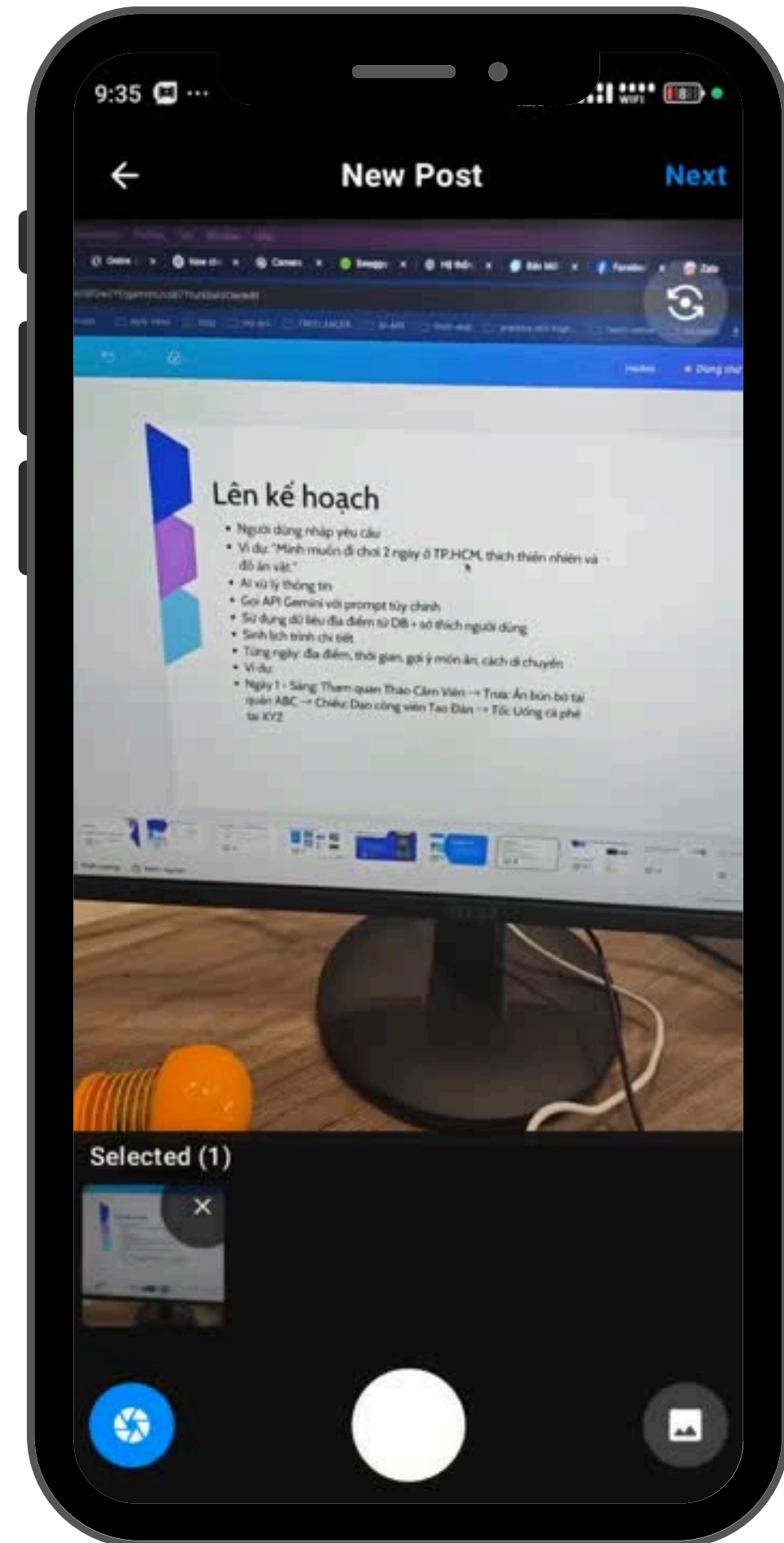
- Hệ thống sử dụng AI để tự động gợi ý lịch trình tối ưu.
- Dựa trên sở thích, thời gian, ngân sách, nhu cầu, ...
- Đưa ra lý do gợi ý giúp người dùng dễ lựa chọn.

=> Giải quyết vấn đề của du khách khi lần đầu đặt chân đến TP Hồ Chí Minh mà chưa có lịch trình cụ thể, ...



Chia sẻ khoảnh khắc

- Người dùng chọn “Chia sẻ khoảnh khắc”
- Nhập nội dung, chọn ảnh/video từ thư viện hoặc chụp mới
- Gắn địa điểm hoặc hành trình liên quan



Triển khai

Tự động hoá với GitHub Actions

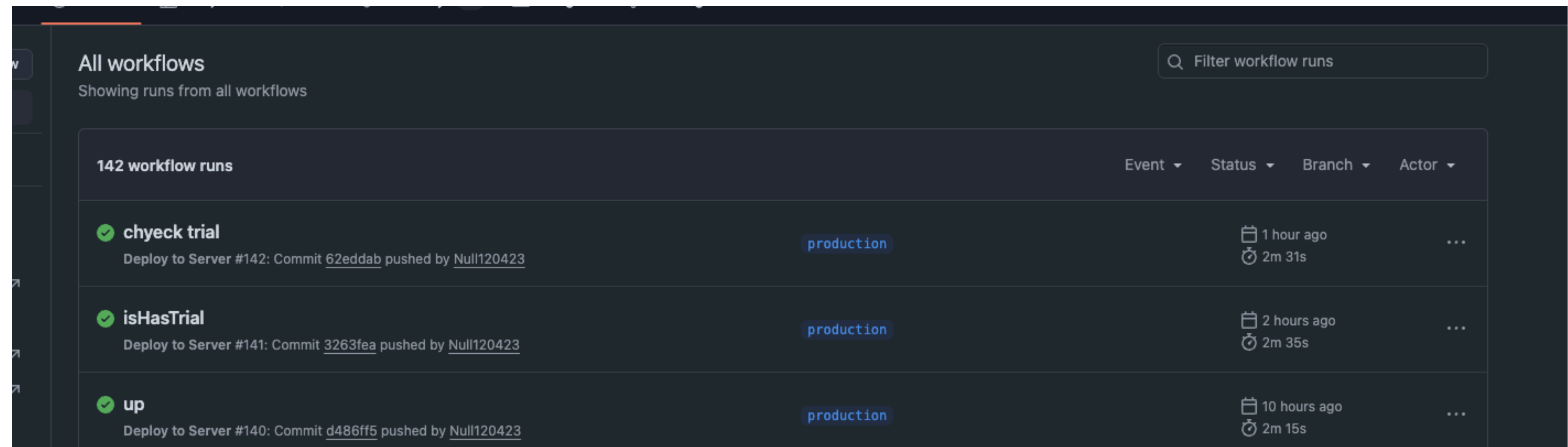
Hệ thống tự động kích hoạt khi có sự kiện push hoặc pull request.

Các bước chính trong quy trình CI/CD:

- Build Docker Image cho từng service
- Đẩy Docker image lên Docker Registry (Docker Hub)
- Deploy lên Cloud Server thông qua SSH hoặc sử dụng shell script
- Restart lại docker container

=> Giúp hỗ trợ khả năng fix lỗi và triển khai tính năng mới nhanh chóng

```
Vaults SFTP LINUX_UB2UB2UUUU +
root@cloud:~# docker ps
CONTAINER ID   IMAGE
D             STATUS        PORTS
2ef46f56a460   huutaidev/hodoshackathon:v1_62eddabb14d67e4980128fa6844293f1768564c3
s ago        Up 2 hours    0.0.0.0:3006->3000/tcp, [::]:3006->3000/tcp    hodoshackathon
3db98a6df0e9   huutaidev/site-demo-seo-freelancer-production:v1_9d335754248875630f8036
rs ago        Up 14 hours   0.0.0.0:3112->3000/tcp, [::]:3112->3000/tcp    site-demo-seo-freelancer-production
99550d78ddfe   huutaidev/hodos-api-schedule:v1_c6a746d9ed6da1c879265fc22e034a6384ba964
rs ago        Up 23 hours   3000/tcp, 0.0.0.0:4000->4000/tcp, [::]:4000->4000/tcp    hodos-api-schedule
915a4417bce9   huutaidev/hodos-admin-production:v1_f8f6be752376201d13d7fe9ec15545bc32d
rs ago        Up 24 hours   0.0.0.0:3008->3000/tcp, [::]:3008->3000/tcp    hodos-admin-production
338793d8cd84   huutaidev/my-profile-production:v1_d23d287605181502e1bf0e3ed54b0266c7c
ago          Up 3 days     0.0.0.0:3111->3000/tcp, [::]:3111->3000/tcp    my-profile-production
7db8fc57908d   postgres
s ago        Up 4 weeks    0.0.0.0:3307->5432/tcp, [::]:3307->5432/tcp    my-postgres
root@cloud:~#
```



Load test và stress test một số API sau khi triển khai với k6

Mục tiêu

- Kiểm tra độ chịu tải của API khi số lượng user tăng dần.
- Đo thời gian phản hồi (response time), tỉ lệ lỗi, và throughput.

Kịch bản test

- Ramp Up: 50 → 100 → 200 → 300 users
- Stress: lên đến 500 users
- Recovery: giảm xuống 100 → 0
- Mục tiêu: <5% lỗi, p(95) response < 1200ms

Công cụ sử dụng

- k6 (Load Testing Tool)
- Output: Terminal Report



Kết quả test

THRESHOLDS

```
http_req_duration
✓ 'p(50)<300' p(50)=238.18ms
✓ 'p(90)<800' p(90)=332.57ms
✓ 'p(95)<1200' p(95)=423.63ms
✓ 'p(99)<2000' p(99)=775.87ms
```

```
http_req_failed
✓ 'rate<0.05' rate=0.00%
```

```
http_reqs
✓ 'rate>50' rate=88.276423/s
```

TOTAL RESULTS

```
checks_total.....: 286782 264.82927/s
checks_succeeded...: 99.98% 286736 out of 286782
checks_failed.....: 0.01% 46 out of 286782
```

```
✓ status is success
✗ response time acceptable
  ↳ 99% — ✓ 95548 / ✗ 46
✓ has data
```

HTTP

```
http_req_duration.....: avg=266.21ms min=97.21ms med=238.18ms max=4.5s p(90)=332.57ms p(95)=423.63ms
{ expected_response:true }...: avg=266.21ms min=97.21ms med=238.18ms max=4.5s p(90)=332.57ms p(95)=423.63ms
http_req_failed.....: 0.00% 0 out of 95594
http_reqs.....: 95594 88.276423/s
```

EXECUTION

```
iteration_duration.....: avg=2.27s min=1.11s med=2.27s max=7.47s p(90)=3.07s p(95)=3.17s
iterations.....: 95594 88.276423/s
vus.....: 1 min=1 max=500
vus_max.....: 500 min=500 max=500
```

NETWORK

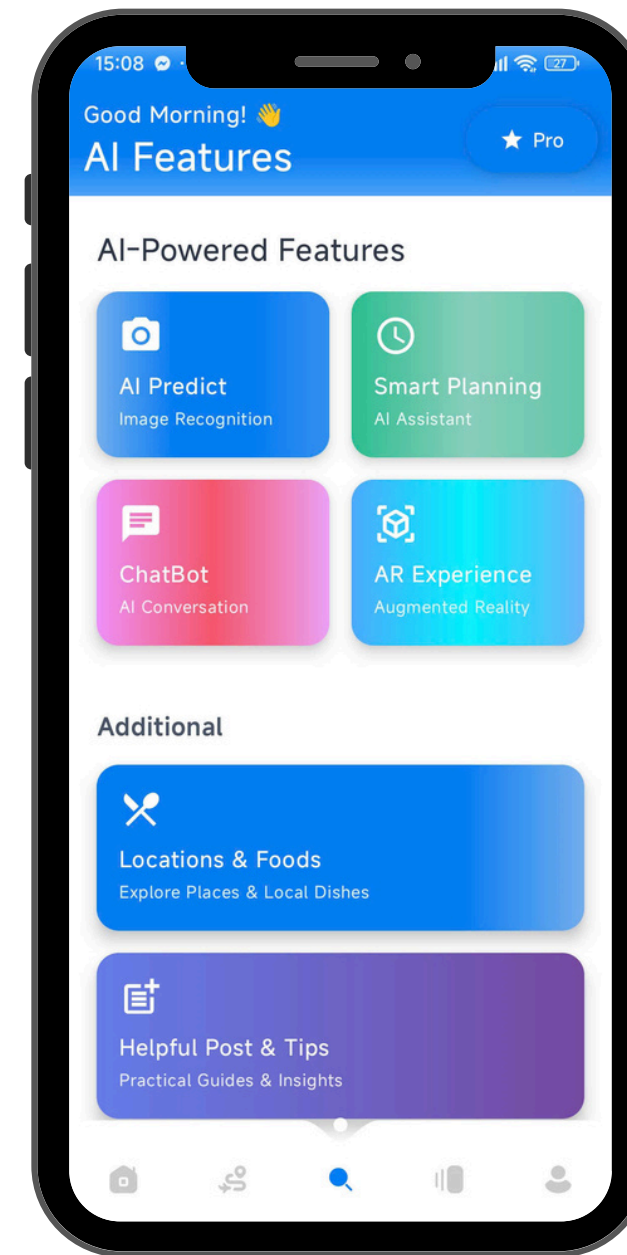
```
data_received.....: 7.7 GB 7.1 MB/s
data_sent.....: 45 MB 42 kB/s
```

```
running (18m02.9s), 000/500 VUs, 95594 complete and 0 interrupted iterations
default ✓ [=====] 000/500 VUs 18m0s
→ benchmark quit:(production)
```

Kết luận

Hoàn thành xây dựng ứng dụng di động hỗ trợ du lịch với các chức năng:

- Tìm kiếm địa điểm, nhận dạng địa điểm và món ăn.
- Gợi ý thông tin thông qua ChatBotAI.
- Đề xuất lịch trình.
- Và một số tính năng phụ khác như cổng Admin để cài đặt và quản lý một vài master data , người dùng, nội dung.
- Triển khai được ứng dụng , web service và một số dịch vụ bên ngoài như mail, tích hợp LLM.
- Load test và stress test đối với một số API sau khi triển khai đảm bảo các chức năng chạy ổn định.
- Stress test với k6 cho thấy hệ thống chịu được tải cao với tỉ lệ lỗi < 5%.



Hướng phát triển

- Phát triển các tính năng liên kết chúng thành một AI Agent phục vụ cho việc du lịch trong suốt chuyến đi của du khách.
- Hoàn thành các tính năng phụ như check in địa điểm, lưu giữ và chia sẻ hành trình cho người thân gia đình.
- Cho phép người dùng tham quan 360 sơ qua về địa điểm.
- Hoàn thành việc đưa ứng dụng lên google play
- Phát triển thêm một version cho IOS

